



NVIDIA Linux For Safety

A System-Level View

igor stoppa istoppa@nvidia.com

ELISA Workshop June 9th - 11th 2026
Co-Hosted by: Canonical



Disclaimers

The present discussion focuses on technology rather than products.

Features and technologies discussed here are to be seen as **demonstrators**, and rather than products/offerings roadmaps.

Similarly, **performance figures should not be inferred**.

The description of **mechanisms** is intentionally **extremely simplified**, to focus on the core concept(s)

Similarly, many other aspects are **intentionally overlooked**, since they are **out of scope** for this presentation.

Agenda

- Recap from previous talks
- NVIDIA safety mechanisms
- System Monitoring
- Conclusion
- Q&A



Recap



[Link to the Video](#)

[Link to the Slides](#)

Address Space Isolation for Enhanced Safety of the Linux Kernel

Igor Stoppa, Sr SW Safety Architect

Version 1.1
September 27th 2022



[Link to the Slides](#)

A Systematic Approach to Using the Linux Kernel in a Safety Scenario

Igor Stoppa <istoppa@nvidia.com>

ELISA Workshop October 17th, 2023



ELISA
Enabling **Linux** in
Safety Applications



WORKSHOP

LUND 2024

[Link to Slides](#)

Overall engineering approach to safety for systems involving Linux

Paul Albertella, Codethink
Igor Stoppa, NVIDIA





Resilient Safety Analysis and Qualification

Igor Stoppa

[Link to the Video](#)

[Link to the Slides](#)



[Link to the Video](#)

[Link to the Slides](#)

Identifying Safety Weaknesses and Fault Propagation in the Linux Kernel

Igor Stoppa | OSSummit Europe 25-27 August 2025

Achieving ASIL B Qualified Linux while minimizing expectations from upstream kernel

(Condensed Edition)

Igor Stoppa <istoppa@nvidia.com>

[Link to the Video](#)

[Link to the Slides](#)

Why Safety with Linux is Hard

The Vanilla Kernel:

- doesn't hold functional safety as a goal
- intentionally prioritizes speed over integrity
- a complex interaction of many complex components
- not well aligned with typical ECU target for ISO standards
- rapidly evolving (1 release every 2 months)
- the kernel can corrupt any driver and any process
 - all cores use the same memory map
 - no internal boundaries

Exhaustive qualification would be necessary,
but it is both infeasible and unsustainable.

The typical approaches

Process-based approach (tailoring)

- Based on indirect methodology
- Assumes ability to cover all the relevant faults
- Impossible to prove sufficiency
- Often “everything is already good”: no actual fixes
- No protection against deficiency in the analysis

Monitoring Harness approach

- Observer “Out of reach” of kernel-initiated failures
- Replaces kernel functionality (Theseus’ Ship)
- Inherently higher latency (round-trip to the harness environment)
- Blind to failure modes in kernel high-performance sections

Userspace Monitor approach

Will be discussed later

NVIDIA Approach

Create “verifiably safe havens” inside the Kernel

- Safety Related Data cannot be silently corrupted/altered
- Retains full direct access to all kernel symbols
- Protects also selected user space components
- Relies exclusively on mechanisms that are qualifiable according to ISO standards.

Enables the creation of safety monitors
and reduces the scope they must support.

Best of Both worlds

The Vanilla Kernel remains as-is

- Fast update cycles where needed
- Mostly non-qualified code, low bureaucracy

Ad-hoc mechanisms provide safety

- Follow well-established ISO processes
- Hardening for the kernel while retaining independence.
- Avoid being stuck on ageing kernel versions.
- Qualify only selected kernel components,
without cascading requirements on the whole kernel.

NVIDIA safety mechanisms



Observation on the Vanilla Kernel

Spatial Interference:

“unauthorized or unintended access, corruption, or overwrite of memory regions belonging to one software element by another element operating at a different ASIL level or safety partition”

Spatial Interference may originate from:

- any **kernel thread** running on **any core**
- any **kernel function** executed at **any time**

Spatial Interference may affect:

- any safety-relevant (SR) **kernel data**
(includes also memory mapped devices)
- any safety-relevant (SR) **userspace process**

What we would like to achieve

Restrict which threads can alter SR data.

Restrict which functions can alter SR data.

Restrict which data must be treated as SR data.

Only selected threads can alter SR data,
both within the kernel and in processes,
and only while executing selected functions.

Only selected data is treated as SR.

Implications

Selective write permission requires multiple kernel memory maps.

Maps activation must be local to each core.

A need for automatic transition mechanism.

A need for data allocation segregation.

The answer: Contexts

Contexts: Diverse in-kernel memory maps:

- Same mapping of address to memory page.
- Different properties of the mappings:
 - Data read operations allowed from any ctx to any other ctx.
 - Data write operations allowed only to same or lesser ctx.

Example: mutual access, with 3 contexts

From \ To → ↓	CTX00	CTX01	CTX02
CTX00	Write: YES Read: YES	Write: NO Read: YES	Write: NO Read: YES
CTX01	Write: YES Read: YES	Write: YES Read: YES	Write: NO Read: YES
CTX02	Write: YES Read: YES	Write: YES Read: YES	Write: YES Read: YES

CTX00: Vanilla Kernel

CTX01: Protected from CTX00

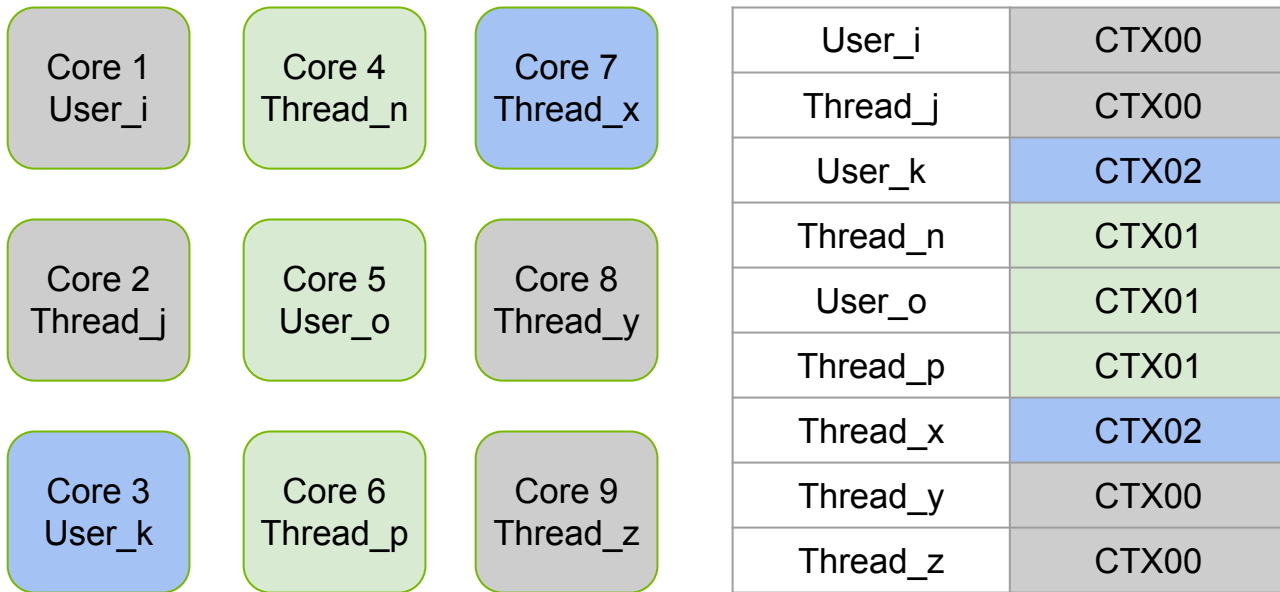
CTX02: Protected from CTX00 AND CTX01

The Per-Thread Aspect

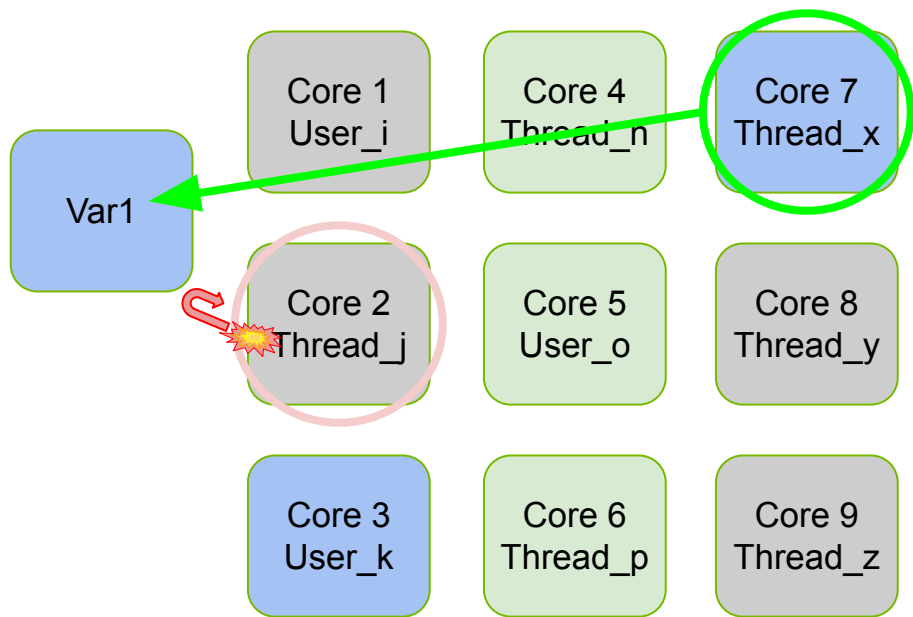
Context-aware scheduler:

- Each thread has an active context.
- It must be restored/saved when:
 - scheduling the thread in/out.
 - migrating the thread to another core.

Example: local activation, with 3 contexts



Example: local activation, with 3 contexts



User_i	CTX00
Thread_j	CTX00
User_k	CTX02
Thread_n	CTX01
User_o	CTX01
Thread_p	CTX01
Thread_x	CTX02
Thread_y	CTX00
Thread_z	CTX00

Both Thread_j on Core 2 and Thread_x on Core 7 are in Kernel mode. However, **Thread_j is unable** to alter memory from **CTX02**, while **Thread_x can**

The Per-Function Aspect

Limiting exposure

A thread running with a given context can alter same or less contexts' data.

But only in few cases it actually needs to do it.

Most of the functions can be run in lower context.

The Per-Function Aspect

Limiting the effort

Any function able to alter SR data must be qualified accordingly to the data safety level.

Avoid whenever possible qualifying upstream code:

- Qualification is costly.
- Upstream functions change frequently.

Transition threads to/from contexts on-need-basis

The Per-Function Aspect

Limiting the maintenance

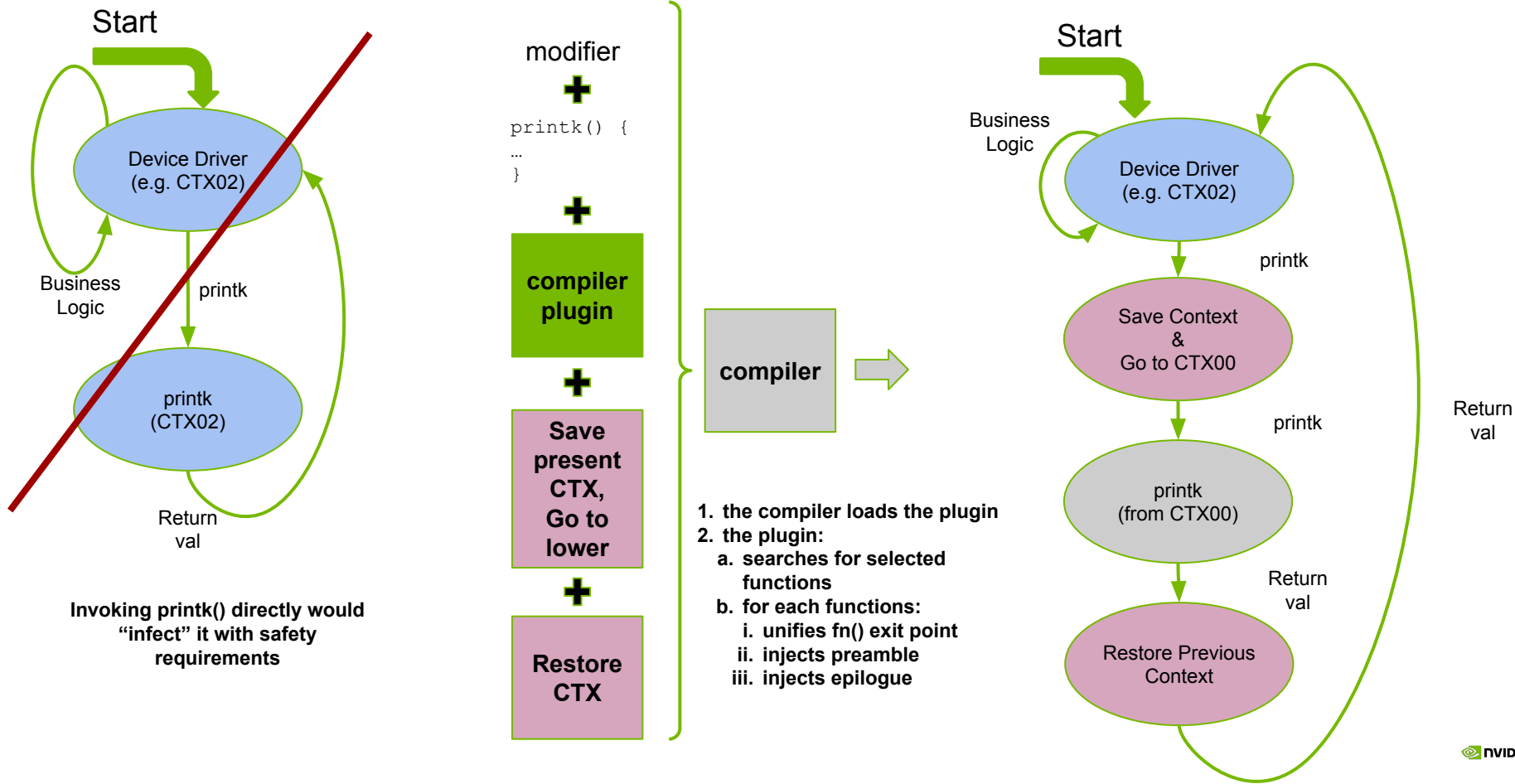
Transitioning a thread to/from contexts requires code.

Goal: avoid very invasive kernel changes.

Solution:

*inject code into upstream functions,
using a compiler plugin.*

Example: avoid qualifying printk



The Data-Allocation Aspect

Marshalling allocations

Given a **certain data object**, we want to specify **if/how**
it should be protected.

It means specifying **which context can alter it.**

In practice **certain memory** will be mapped to be
writable by a given context but not by lesser ones.

The “certain memory” is a **context-specific “pool”**

The Data-Allocation Aspect

Marshalling allocations

Contexts rely on MMU mapping ability.

MMU mappings uses page-size granularity.

A pool is a collection of pages reserved for a context

The allocators in the vanilla kernel
don't support selective pool allocation.

Default allocators must be augmented.

The Data-Allocation Aspect

Avoiding qualifying the Kernel memory managers

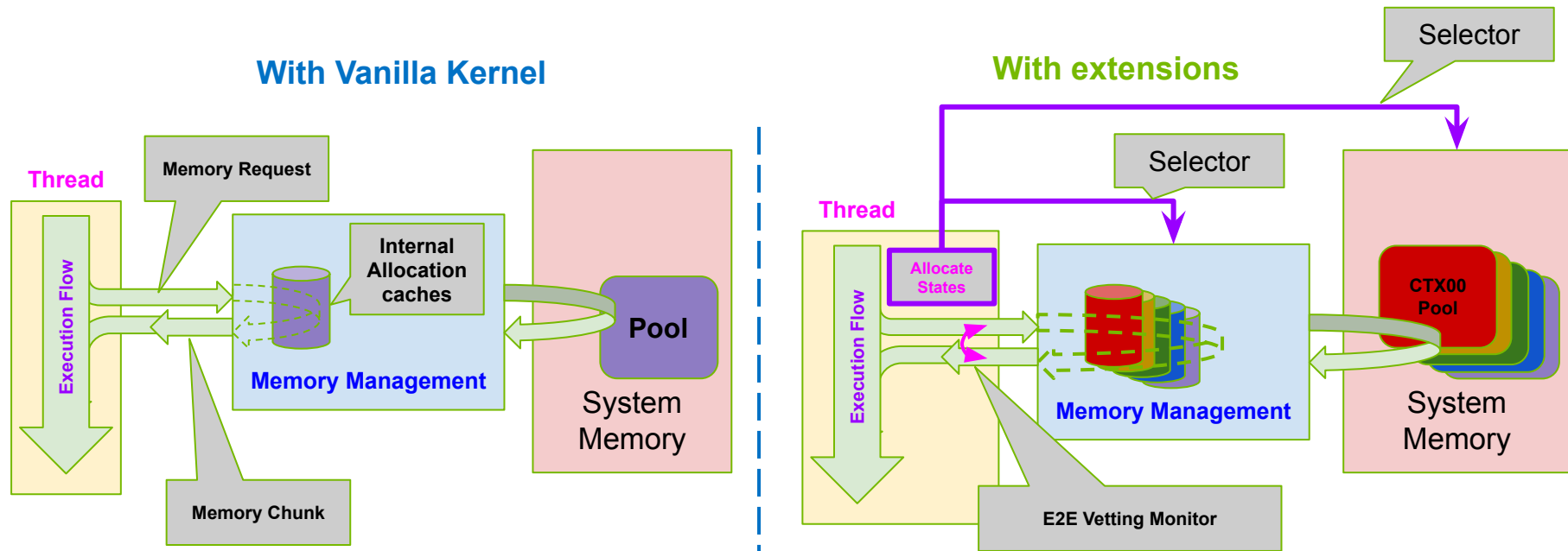
Kernel memory managers are complex state machines.

True qualification is both unrealistic and unfeasible.

It's better to leave them as non-qualified.

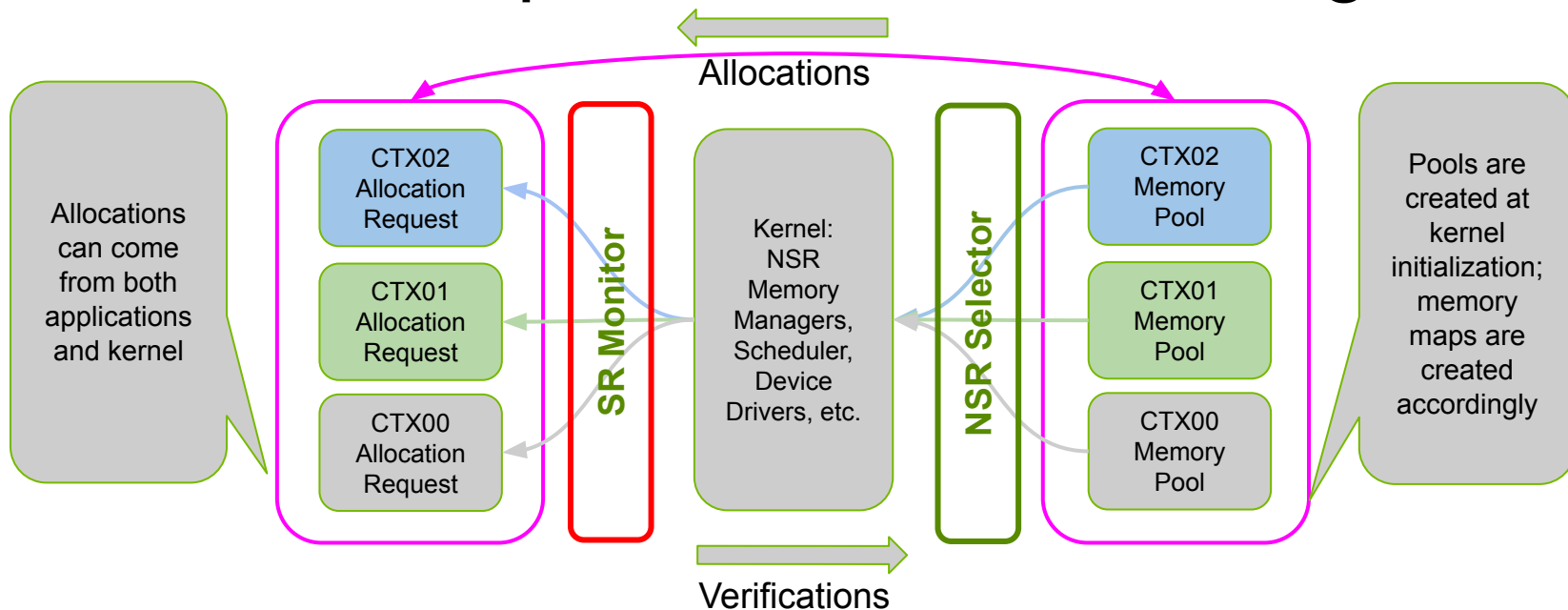
Use qualified End-To-End vetting, instead.

Example: Marshalling Allocations



- A memory allocation uses memory from a pool selected based on the state of thread requesting the memory allocation.
- Prior to releasing the memory to the thread, it is vetted for correctness.
- The memory managers involved do not need to be qualified.

Example: End-to-end Vetting



- Only the SR Monitor performing the End-to-end vetting needs to be qualified

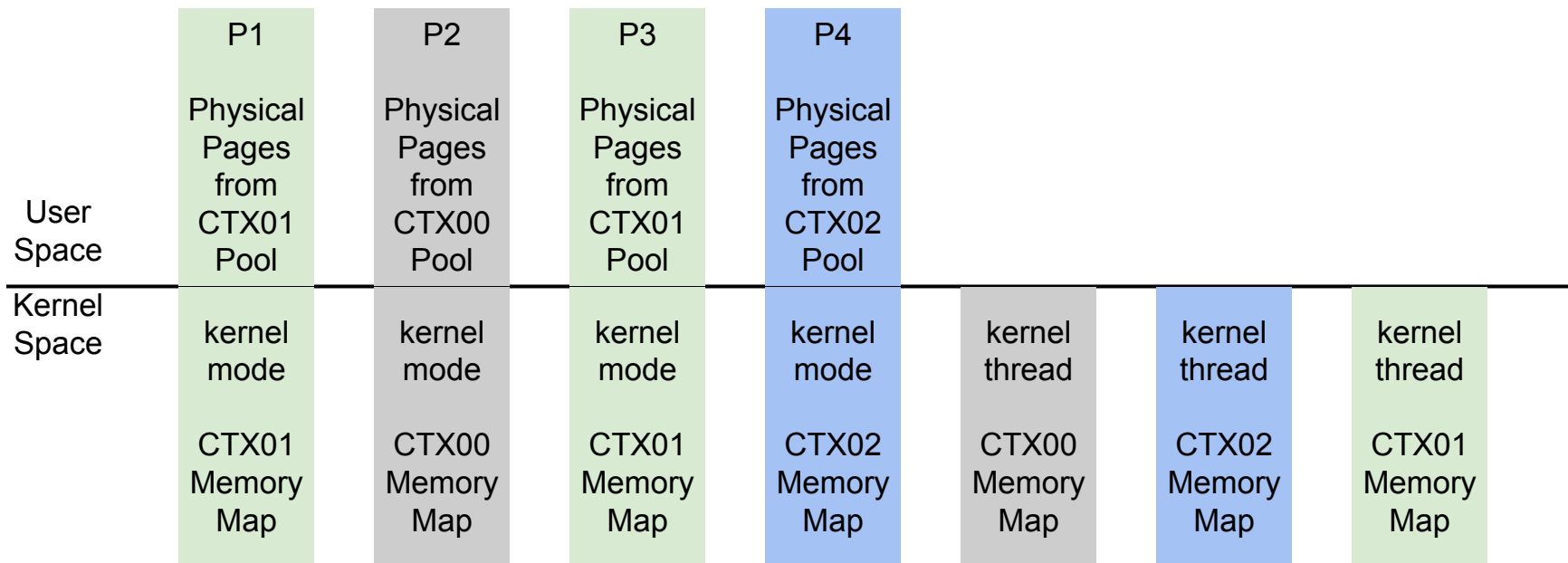
The Process-Creation Aspect

Userspace Processes follow the same flow

The Kernel treats processes as if they were data,
when it creates them.

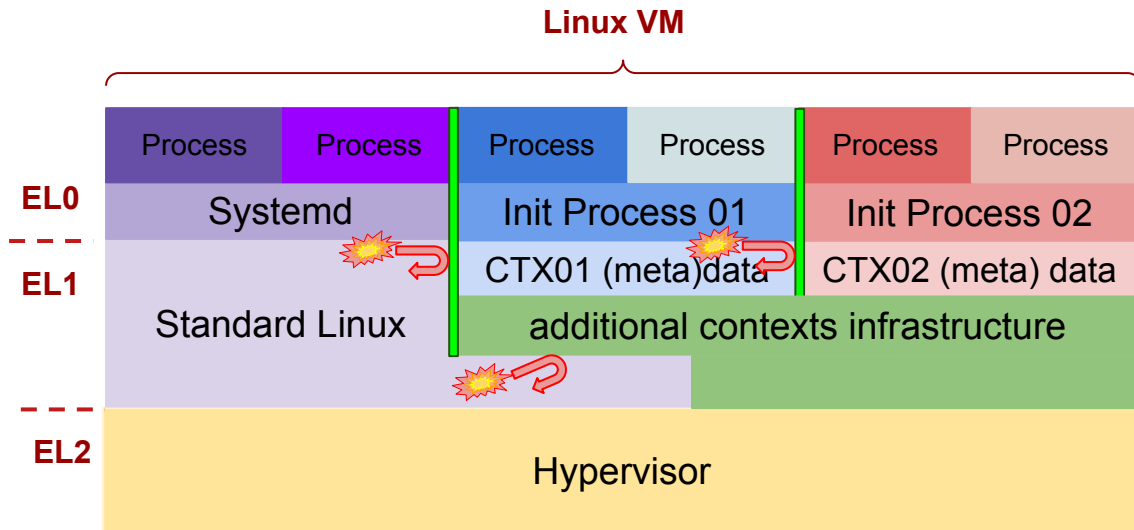
The same allocation strategy works also for creating
higher-context processes that are shielded from
kernel threads running in lesser context.

Example: kernel and userspace threads



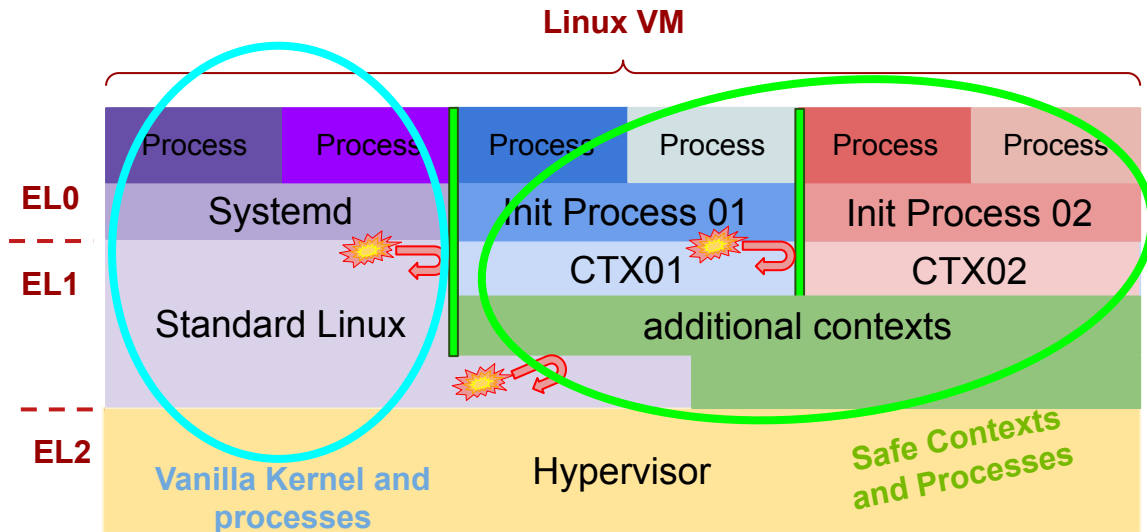
Memory for the userspace-side of processes is allocated from the associated contexts.

Example: Spatial Isolation



The MMU prevents rogue write operations from succeeding.

Example: Spatial Isolation



The MMU prevents rogue write operations from succeeding.

The Approach to Safety

Partitioning the system

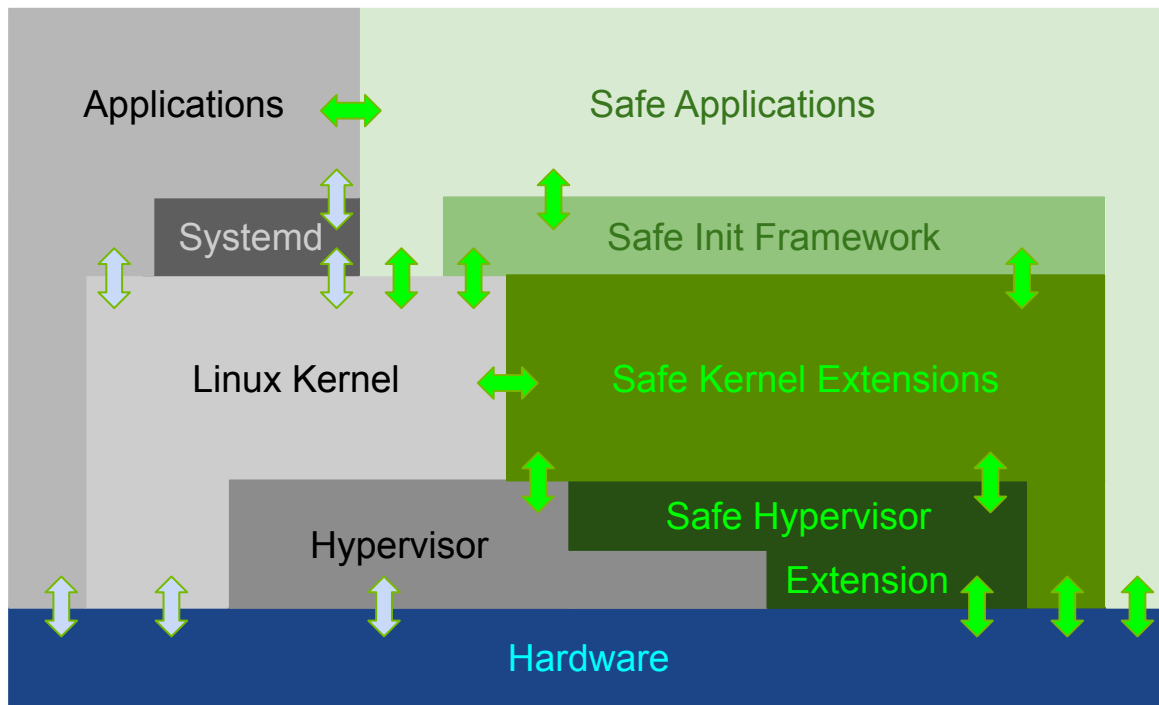
Avoid penalizing performance of NSR components.

Provide strong protection to what must be SR.

Treat as SR only what is realistically qualifiable.
(e.g. leave systemd as NSR)

Bring online SR components prioritised by safety level.

A Comprehensive Software Stack



↕ Interaction between NSR Components

↕ Interaction with a SR Component

Safe Components interact with both each other and with other non-safe ones.

System Monitoring

Reacting to Anomalies

Most concepts shown so far relate to “space”

But there is also a temporal dimension that must be accounted for ...

- provide timely reaction to SR stimuli
- react promptly to violations of SR constraints

Dealing with Time

Temporal Interference:

“the timing behaviour of one software element disrupts the timing requirements of another element, leading to a potential violation of safety-related timing constraints and, ultimately, a safety goal”

Temporal Interference may originate from:

- incorrect prioritization (by the kernel) of SR activities
- inability to satisfy needs of SR components

The typical approach is to use some form of watchdog.

Monitoring Pitfalls

Using a watchdog might appear simpler than it actually is.

Dependency aspects:

- pinging must succeed **only when the system is not starved**
FAIL example: an SR process is starved, but the thread pinging the WD
- pinging must succeed **only when all the SR parameters are healthy**
FAIL example: some SR parameter is omitted from the dependency
FAIL example: some SR parameter is altered in an undetected way

Monitoring Pitfalls

Using a watchdog might appear simpler than it actually is.

Root-of-Safety/Chaining aspects:

- monitoring must have its roots in something **immune from SW interference**
FAIL example: a Kernel and/or Userspace monitor without a HW WD
- each layer must use **semantics able to express its safety requirements**
FAIL example: use simple pinging when loss of synchronisation is possible

Monitoring Pitfalls

Using a watchdog might appear simpler than it actually is.

Integrity/Correctness aspects:

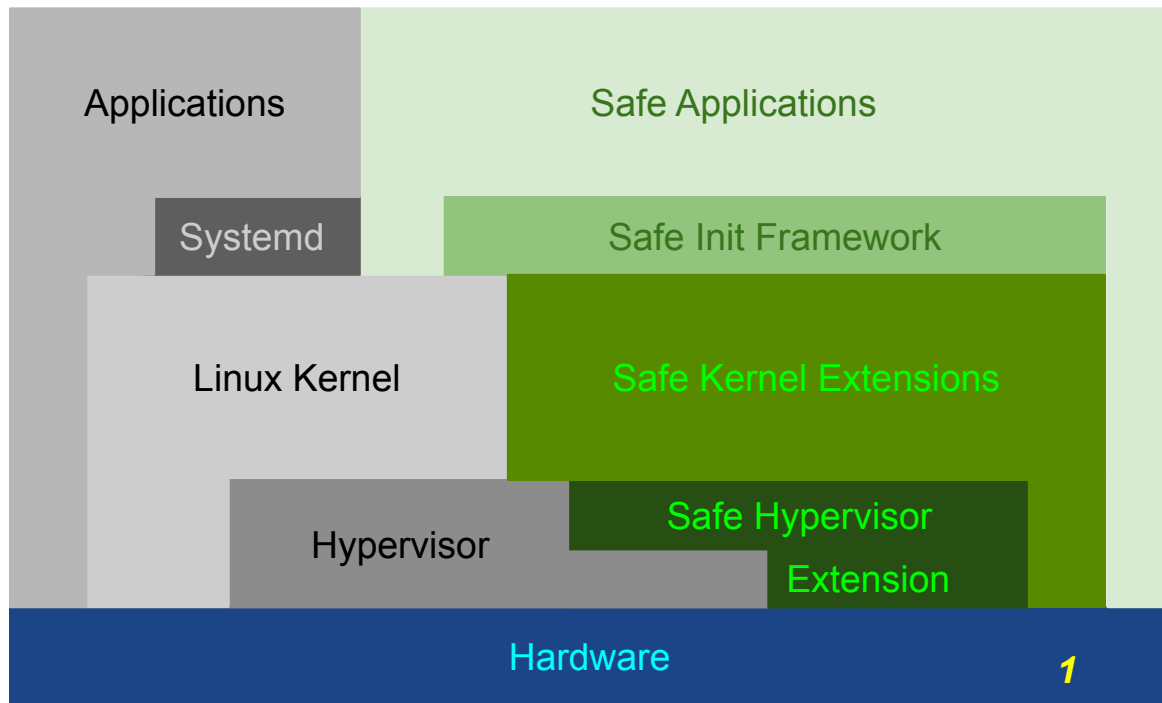
- using Rust (or any other provable approach) for the monitor/monitored, does not prevent interference from the kernel to their internal states
- relying on redundancy for protecting internal states presents both a completeness and a coherence problem
All the relevant states must be hardened AND vetted at each R/W access

Reliable Safe Monitoring

Create a hardened chain-of-safety with a HW WD as root-of-safety.

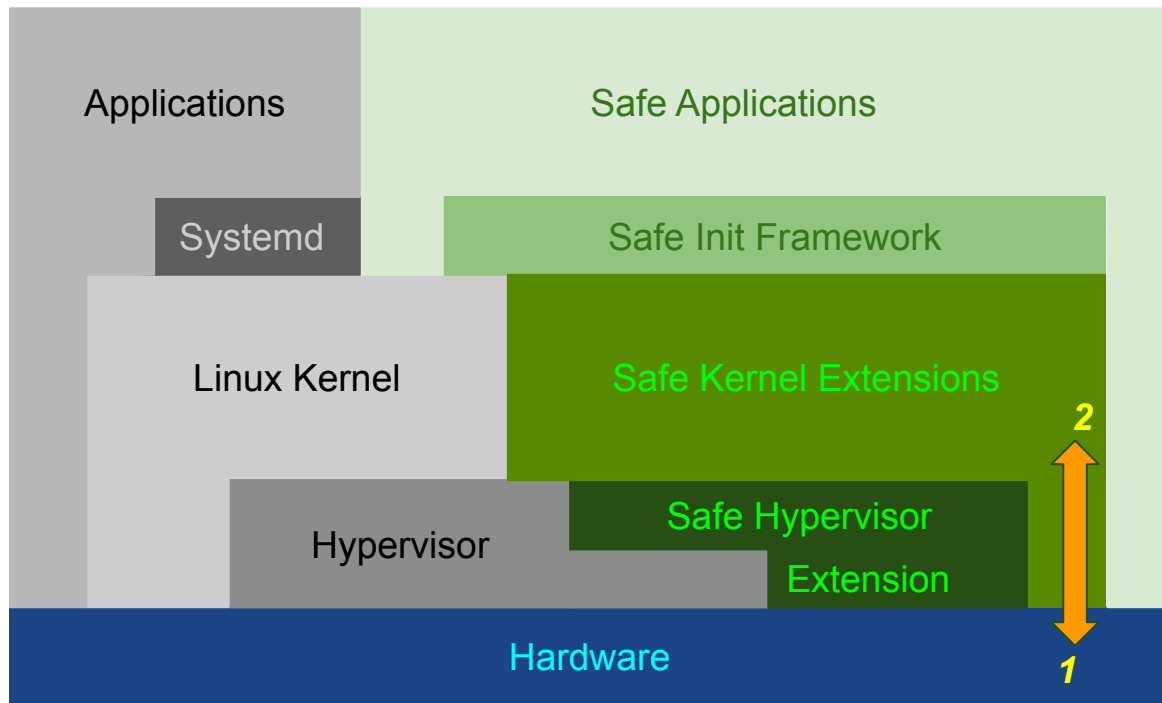
- each layer uses semantic compatible with its level of abstraction
- the cascading terminates in a HW component immune to SW interference
- monitoring is performed from within the system, for maximum observability
- monitoring is protected from interference through isolation
- isolation through HW mechanisms, able to provide large enough blanket

A Chain of Safety



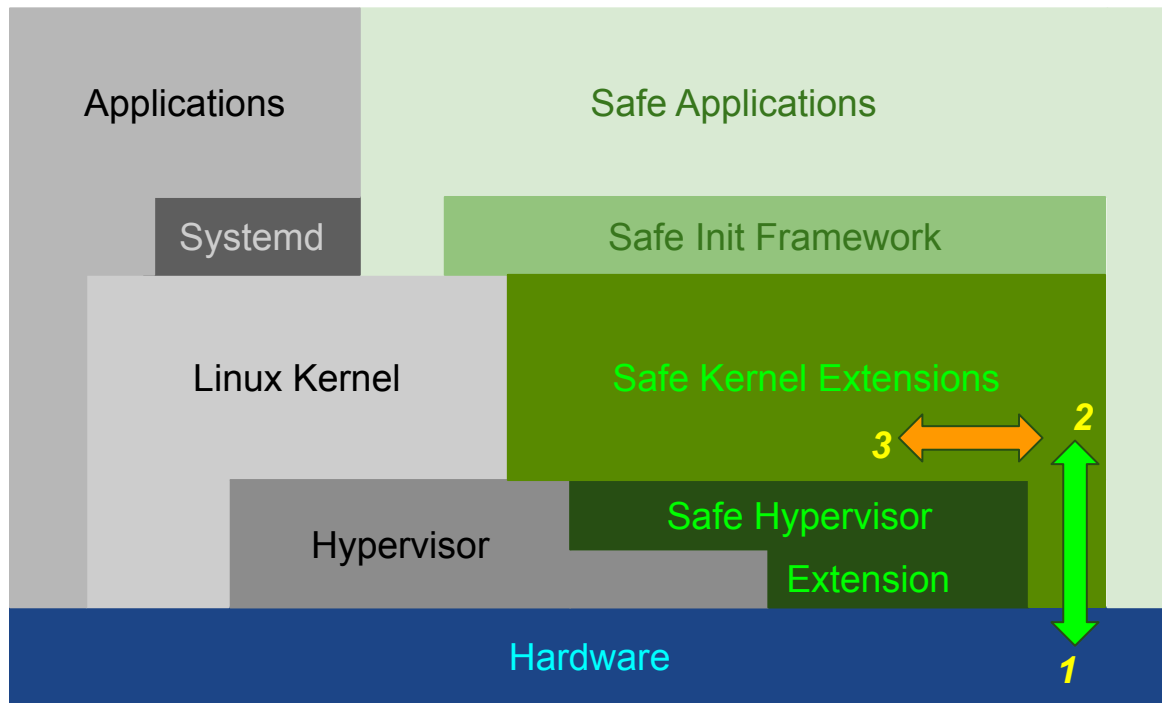
- Need for a HW-based **“safety anchor”** (1)

A Chain of Safety



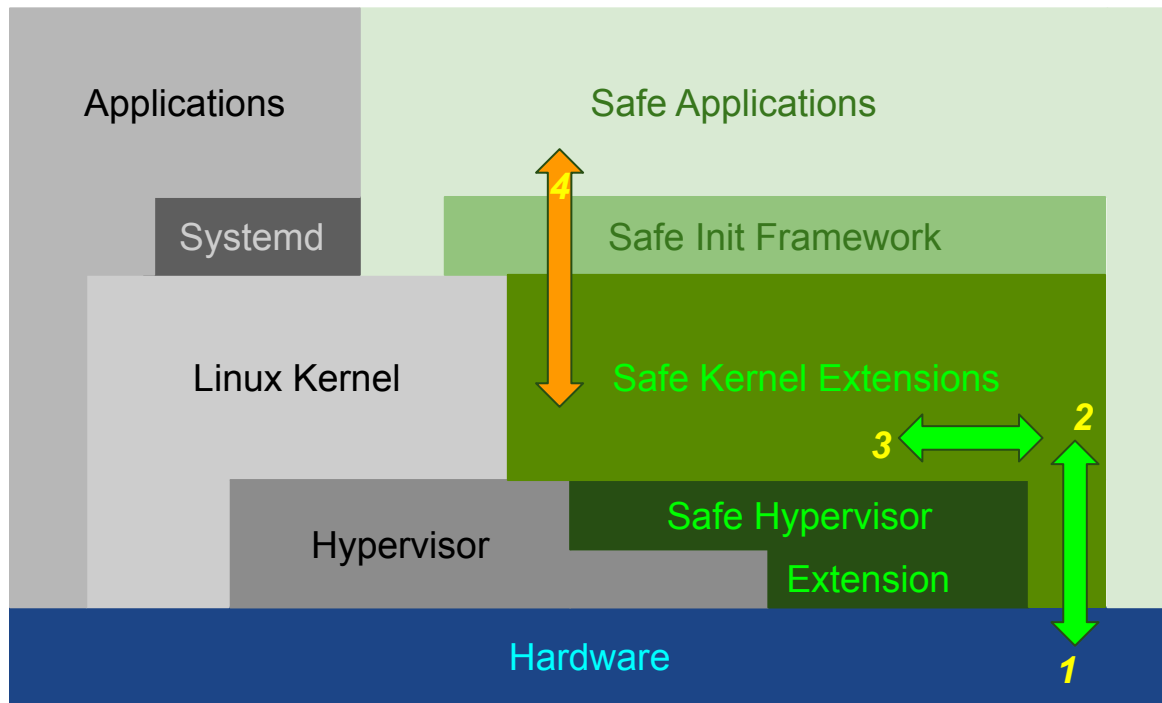
- Need for a HW-based “safety anchor” (1)
- Core In-Kernel Monitoring relies on the “safety anchor” (2)

A Chain of Safety



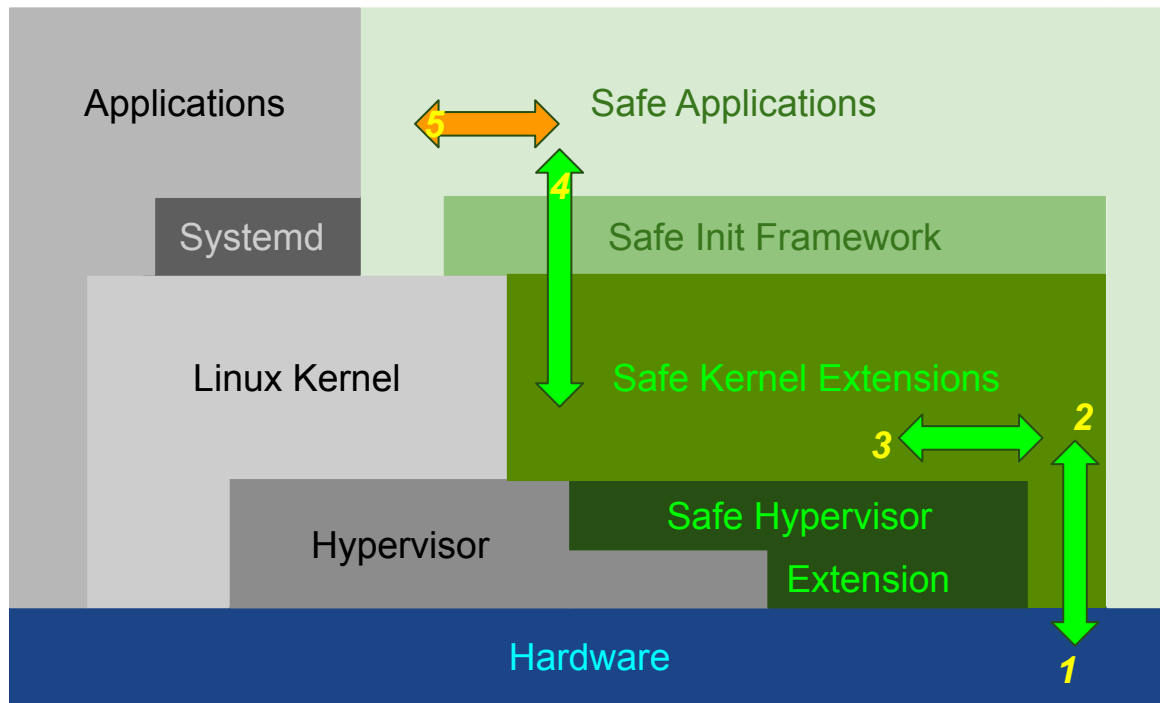
- Need for a HW-based “safety anchor” (1)
- Core Kernel relies on the “safety anchor” (2)
- Kernel components must be monitored as well (3)

A Chain of Safety



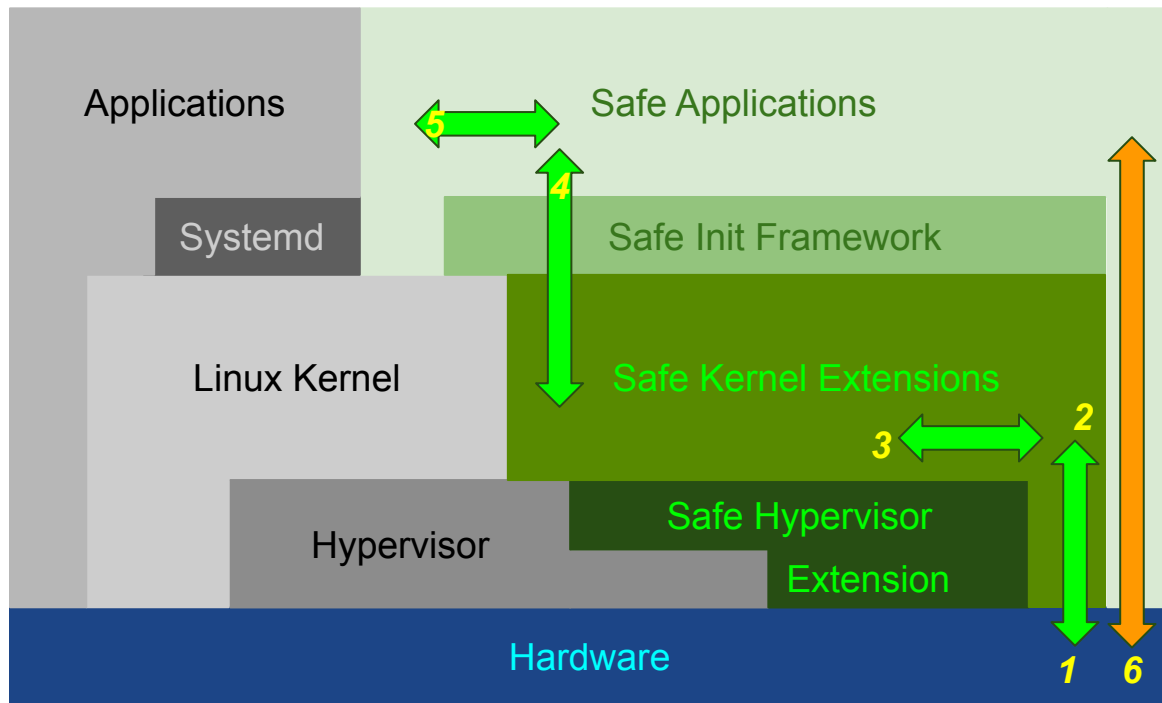
- Need for a HW-based “safety anchor” (1)
- Core Kernel relies on the “safety anchor” (2)
- Kernel components must be monitored as well (3)
- User space functionality falls into different classes and might require monitoring from:
 - kernel (4)

A Chain of Safety



- Need for a HW-based “safety anchor” (1)
- Core Kernel relies on the “safety anchor” (2)
- Kernel components must be monitored as well (3)
- User space functionality falls into different classes and might require monitoring from:
 - kernel (4)
 - ad-hoc processes (5)

A Chain of Safety



- Need for a HW-based “safety anchor” (1)
- Core Kernel relies on the “safety anchor” (2)
- Kernel components must be monitored as well (3)
- User space functionality falls into different classes and might require monitoring from:
 - kernel (4)
 - ad-hoc processes (5)
 - Hardware (6)

Conclusion

Wrapping it Up (for now ...)

The Vanilla Linux Kernel is both exposed to and can generate both Spatial and Temporal Interference

Managing Spatial Interference in a verifiable way without partitioning has a **completeness problem**.

HW-backed partitioning is the only realistic way, given the **surface to protect**.

In-Kernel monitoring is necessary, to not miss failures without losing performance.

In-Kernel partitioning is necessary, to **protect the in-kernel monitor**.

Temporal monitoring without shielding from Spatial Interference has a **completeness problem**.

Q&A

Thank You

